

郵件作業服務系統 FOR Ishop

MailTaskJobService 使用說明

功能概述

MailTaskJobService 是一個獨立運行的郵件任務處理服務，它定時從 **IshopX** 平台（或任何實現了相同 API 的郵件任務平台）拉取待發送的郵件任務，並使用 **MailEnhanceService** 完成郵件發送。發送完成後自動回呼平台更新任務狀態，支援多租戶、並發控制、失敗重試等特性。

系統架構

```
1 MailTask••JobService (計時執行)
2 |
3 |─ 1. 呼叫 IshopX API: GetMailTask••JobRequest
4 | ↓
5 | 取得郵件任務 (收件者清單、範本內容等)
6 | ↓
7 | 2. 呼叫 updateTaskOccupied 標記任務為“佔用中”
8 | ↓
9 | 3. 使用 MailEnhanceService 傳送郵件 (支援多帳號輪詢、SSL/TLS)
10 | ↓
11 | 4. 呼叫 updateEmailTask••JobComplete 標記任務為“已完成”
12 |
13 |─ 重複以上步驟 (依配置間隔)
```

與 IshopX 平台的請求流程

1. 取得郵件任務 (GetMailTaskJobRequest)

- 請求位址: `{MailDataApiUrl}/mgr/ShopMailService/GetMailTaskJobRequest`
- 請求方法: `POST`
- 請求頭: `Authorization: Bearer {AuthenticationCode}`
- 請求體: `application/x-www-form-urlencoded`，欄位 `shopId={店舖ID}`
- 回應範例:

```
1 {
2   "meta": { "Success": true, "Message": "取得郵件任務成功", "ErrorCode": 0
3 },
4   "data": {
5     "Success": true,
6     "EmailTask.Id": "1734567890123",
7     "ShopId": "sh0006",
8     "CallbackUrl": "",
9     "ToMailList": [
10      { "Email": "user@example.com", "Name": "張三" }
11    ],
12     "Subject": "促銷活動",
13     "EmailBody": "base64編碼的HTML內容"
14   }
15 }
```

2. 🌟 標記任務佔用 (UpdateTaskOccupied)

- 目的: 防止多個服務實例同時處理同一個任務。
- 請求位址: {MailDataApiUrl}/mgr/ShopMailService/UpdateTaskOccupied
- 請求方法: POST
- 請求頭: Authorization: Bearer {AuthenticationCode}
- 請求體: shopId={店鋪ID}&emailTaskId={任務ID}

3. 🌟 標記任務完成 (UpdateEmailTaskJobComplete)

- 請求位址: {MailDataApiUrl}/mgr/ShopMailService/UpdateEmailTaskJobComplete
- 請求方法: POST
- 參數: 相同佔用接口
- 作用: 將 EmailTask 狀態改為 INACTIVE，同時將關聯的 EmailSubscribe 狀態改為 INACTIVE（表示這些訂閱者已發送過該任務，避免重複發送）。

📌 配置說明 (appsettings.json)

📌 必填設定項

```
1 // IshopX web App
2 // 郵件數據API的URL地址，定時作業服務將通過此URL地址從郵件數據API獲取郵件發送所需的數據"
3 // 定時作業服務從郵件數據API獲取數據的時間間隔，單位為分鐘。建議根據實際需求設置合理的時間間隔，避免過於頻繁地請求API導致性能問題。
4 // 測試 使用每隔 1分鐘執行，其它情況，越長越好，建議 180分鐘以上，具體根據業務需求和郵件發送頻率來決定。
5 "MailTaskServiceSetting": {
6   "MailDataApiUrl": "http://localhost",
7   "IntervalMinutes": 5,
8   "MailToDefault": "mcessol2000@gmail.com" //默認發送目標郵箱地址
9 },
```

對應的對象是 MailTaskServiceSetting

配置節	說明
MailDataApiUrl	IshopX 平台的基礎 URL (例如 <code>https://xguard.hk</code>)
AuthenticUser	認證使用者數組，支援多租用戶。每個物件包含： - <code>mainComId / shopId</code>：租用戶識別碼 (二選一，優先使用 <code>shopId</code>) - <code>authenticationCode</code>：Bearer Token - <code>authenticMethods</code>：固定 <code>"Bearer"</code>
MailToDefault	當收件者清單為空時所使用的測試信箱
senderEmailAccountList	寄件帳號清單 (可設定多個，用於輪詢傳送) 也可透過 <code>./Config/SenderEmailAccountList_{mainComId}.json</code> 覆蓋

📌 Kestrel 配置 (用於自架)

如果希望服務同時提供健康檢查或自訂管理接口，可設定 `kestrel` 端點。預設不需要 Web 端口，僅透過定時任務運行。

完整設定範例

```
1 {
2   "MailDataApiUrl": "https://xguard.hk",
3   "AuthenticUser": [
4     {
5       "mainComId": "6000014",
6       "shopId": "sh0006",
7       "userName": "XGuard1",
8       "authenticationCode": "0f18abd25e9519cd69b0b4848fa393fd",
9       "authenticMethods": "Bearer"
10    }
11  ],
12  "MailToDefault": "mcesso12000@gmail.com",
13  "senderEmailAccountList": [
14    {
15      "senderOfCompany": "xguard.hk",
16      "mailTool": 1,
17      "enableSSL": false,
18      "enableTSL": true,
19      "senderServerHost": "smtp-relay.brevo.com",
20      "senderServerHostPort": 587,
21      "fromMailAddress": "service1@xguard.hk",
22      "fromMailDisplayName": "Service Center Of Xguard",
23      "senderUserName": "955ced001@smtp-brevo.com",
24      "senderUserPassword": "your-smtp-key",
25      "Remarks": "Brevo SMTP"
26    }
27  ]
28 }
```

如何運行 MailTaskJobService

1. 環境要求

- .NET 8.0 Runtime
- Windows / Linux / macOS
- 網路可存取 IshopX 平台的 API 位址

2. 📁 部署文件

將以下檔案放在同一目錄下：

- MailTaskJobService.exe (或 Linux 下的 MailTaskJobService.dll)
- appsettings.json (設定檔)
- log4net.config (日誌配置)
- Cer 資料夾 (如果使用客戶端憑證)
- Config 資料夾 (可選，用於存放租戶專屬寄件帳號配置)

3. 📁 運行方式

方式一：直接執行控制台程序

```
『 `bash
```

Windows

MailTaskJobService.exe

Linux / macOS

dotnet MailTaskJobService.dll

```
1
2 首次運行會顯示 `[Start Mail Task Job Service]` 並啟動 Quartz 定時任務，預設執行間隔由 Quartz 配置決定 ( 程式碼中硬編碼為 `withCalendarIntervalSchedule`，可自行修改或設定化 )。
3
4 ##### 方式二：作為 windows 服務安裝
5
6  『 `bash
7  sc.exe create MailTask●●JobService
   binPath="C:\path\to\MailTask●●JobService.exe"
```

方式三：使用 Docker

```
1 FROM mcr.microsoft.com/dotnet/runtime:8.0
2 WORKDIR /app
3 COPY . .
4 ENTRYPOINT ["dotnet", "MailTask.dll"]
```

建置並運行：

```
『 `bash
```

```
docker build -t mailtaskjobservice .
```

```
docker run -d --name mailtask -v ./appsettings.json:/app/appsettings.json
mailtaskjobservice
```

```
1
2 ### 4. 日誌查看
3
4 - 預設日誌輸出到控制台。
5 - 如果配置了 `log4net.config`，日誌會寫入 `Log/` 資料夾，按日期滾動。
6 - 建議透過日誌監控任務執行情況。
7
8 ## 注意事項
9
10 ### 1. 🛡️ 認證安全
11
12 - `authenticationCode` 必須與 IshopX 平台
13   `./Config/EmailWebAppService_{shopId}.json` 中的 `AuthenticationCode`
14   **完全一致** 。
15
16 - 請勿將 Token 提交至公用程式碼倉庫，建議使用環境變數或金鑰管理服務。🔑
17
18 ### 2. 🚦 並發任務控制
19
20 - 服務在取得任務後會立即呼叫 `updateTaskOccupied` 將任務狀態置為 `OCCUPIED`，
21   防止叢集中多個實例同時處理相同任務。
22 - 如果發送程序異常退出，平台上的任務會保持在 `OCCUPIED` 狀態，需要管理員手動恢復
23   ( 或設定逾時自動重置 ) 。
24
25 ### 3. 📧 寄件帳號設定優先權
26
27 - 如果 `./Config/SenderEmailAccountList_{mainComId}.json` 存在，將覆蓋
28   `appsettings.json` 中的 `senderEmailAccountList` 。
29 - 這樣可以為不同租戶 ( 不同 `mainComId` ) 分配不同的寄件帳號池。
```

```

25 ### 4. 📁 郵件範本路徑
26
27 - `EmailTemplate` 欄位儲存的是相對於網站根目錄的路徑 ( 例如
  `~/Upload/Template/xxx.html` ) 。
28 `MailTask` 透過 `Server.MapPath` 解析為實體路徑，**要求服務執行
  時能存取到該路徑** 。
29 - 建議將範本檔案放在共用目錄或複製到服務本機 `MailTemplate` 資料夾下，並相應調整路
  徑。
30
31 ### 5. 📧 收件者管理
32
33 - 每個 `EmailSubscribe` 記錄可以關聯多個 `EmailTask` 。
34 一旦任務完成，該 `EmailSubscribe` 的 狀態會被置為 `INACTIVE`，表示該訂閱者不會
  再被後續任務發送。
35 - 如需支援多次傳送相同訂閱者，應修改業務邏輯：使用獨立的發送記錄表
  ( `EmailSendHistory` )，而不修改 `EmailSubscribe.Status`。
36
37 ### 6. 🛠 故障排除
38
39 | 問題 | 可能原因 | 解決方法 |
40 | ----- | ----- | ----- |
41 | 無法取得任務 | `MailDataApiUrl` 錯誤，或網路不通 | 檢查 URL 是否包含
  `/mgr/ShopMailService/GetMailTask`，測試 `curl` 存取 |
42 | 認證失敗 | `authenticationCode` 不一致 | 核對平台設定檔中的 Token |
43 | 郵件傳送失敗 | SMTP 設定錯誤、連接埠不通 | 檢查 `senderEmailAccountList` 中
  的伺服器位址、連接埠、SSL/TLS 設定 |
44 | 任務卡在 `OCCUPIED` | 傳送程序當機 | 手動將資料庫 `EmailTask.Status` 改回
  `ACTIVE` |
45
46 ### 7. 📊 效能建議
47
48 - 建議將 `HttpClient` 重複使用 ( 程式碼中每次建立新實例，可最佳化為
  `IHttpClientFactory` ) 。
49 - 對於大量收件者的任務，可考慮分批傳送並更新進度，避免長時間阻塞定時任務。
50
51 ## 自訂定時任務間隔
52
53 在目前版本中，Quartz 任務使用 `withCalendarIntervalSchedule` 並固定為
  `intervalMinutes` 參數。如需調整，請修改以下程式碼並重新編譯：
54
55 ```csharp
56 // MailTask.ServiceJOB.cs 中的 RunGlobalProgram 方法

```

```
57 // 將 intervalMinutes 改為從設定檔讀取
```

或使用 `CronScheduleBuilder`：

```
1 | .withCronSchedule("0 */5 * * * ?") // 每5分鐘執行一次
```

常見問題

Q: 為什麼不直接呼叫 `Run` 方法，而是使用 `Quartz`？

A: 為了支援長時間運行和定時輪詢，`Quartz` 提供了可靠的調度和重試機制。

Q: 能否同時處理多間店鋪的任務？

A: 可以。`AuthenticUser` 陣列支援多個租用戶，`BatchListRun` 會遍歷每個租用戶依序呼叫。

Q: 發送郵件失敗會重試嗎？

A: `EmailEnhanceHelper` 內部支援多帳號輪詢，但不會自動重試單一收件者。失敗記錄會記錄在日誌中。

技術支持

如有問題，請查看日誌檔案或聯絡系統管理員。